

Object-oriented programming concepts

Jarmo Hurri

Introduction

- initial look at object-oriented programming
- ideas, terminology, (little) code, models
- large topic that will be covered over a long period of time

Some real-world datatypes

- company access control and working hour monitoring
 - employee, guard, manager
 - access card, permission, group
 - door, lock, card reader, sensor
- mathematics
 - constant, symbol
 - operator, expression, function
 - equation, inequality
 - polynomial, rational function
- banking
 - account, loan
 - application, investment
 - customer, service point, meeting

Example 1: class

Product
description: str id_num: int price: float
<code>__init__(description : str, id_num : int, price : float)</code> <code>__str__(): str</code>

- starting point of object-oriented programming
- defines a data type
- *object* is an *instance* of a class, just like 42 is specific integer
- let us now create this class and some objects
- note: in Python some methods are called *automatically*
 - `__init__()` when object is initialized
 - `__str__()` when user-friendly string representation is needed

Example 2: automatically assigned unique id

- what if we want `id_num` to be created automatically (running integer starting from 0)?
- for this we can use *static variable* (*class attribute*)
 - shared by all instances (objects) of class
 - first bound within class block outside methods
 - accessible inside instance methods as `ClassName.identifier`

Class definition in Python

- basic form

```
class ClassName:
    body
```
- PEP 8 recommends user-defined classes to
 - begin with capital letter
 - follow CapWords convention
- attribute `x` bound in body of class *outside methods*
 - is *class attribute* in Python terminology
 - is *static variable* in IB terminology
 - can be referred to as `ClassName.x` in instance methods (IB: non-static methods)

Special (dunder) methods in Python

- particular feature of Python
- *special methods* called automatically when required
 - also called *dunder* (double underscore) methods
- some examples

method	called when
<code>__init__()</code>	object initialized
<code>__str__()</code>	user-friendly string form needed (e.g. <code>print()</code>)
<code>__add__()</code>	object added to another

- for example, the following are equivalent

```
print(1 + 2)
print((1).__add__(2))
```

- typically we only need to define `__init__()`

Classes in general

- provide new level of abstraction
- contain both data and associated functionality (methods)
- “user-defined datatypes with functionality”
- can have relationships, some of which are supported by programming languages (e.g., inheritance)
- hide irrelevant details and complexity
- like procedures, can be
 - used to structure programs into meaningful modules
 - reused
 - used to divide work
- instance has a lifecycle from creation to destruction as object

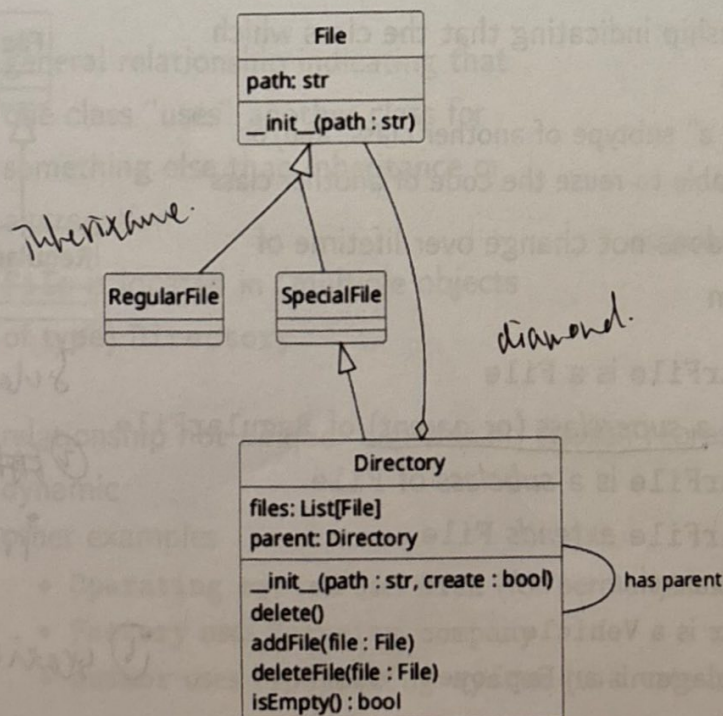
Terminology forest

Python	general oop
attribute	member
method	member function
"field" (not common)	member variable / data member
class attribute	static member
class method (later)	"static member function" (not exactly)
static method (later)	static member function
instance attribute	non-static member

© Jarmo Hurri

8/19

Example 3: simplified file system



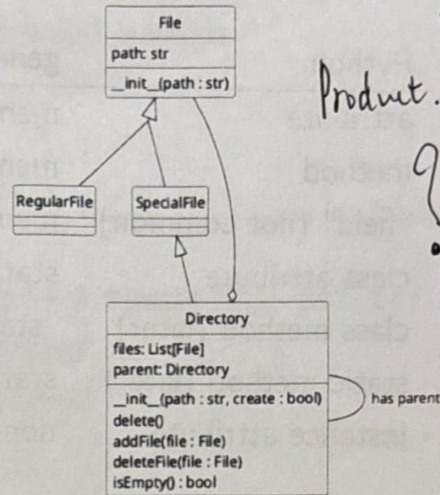
© Jarmo Hurri

9/19

Unified Modeling Language (UML)

relationship among classes. — draw.

- visualization of program structure and functionality
- different types of structural and behavioural diagrams
- (for our purposes) the most relevant of these is the *class diagram*
- depicts classes (with or without internals) and their relationships



for example:

p. { 属性的。
belonged to object.

Product. mro()

python automatically makes it
class → object

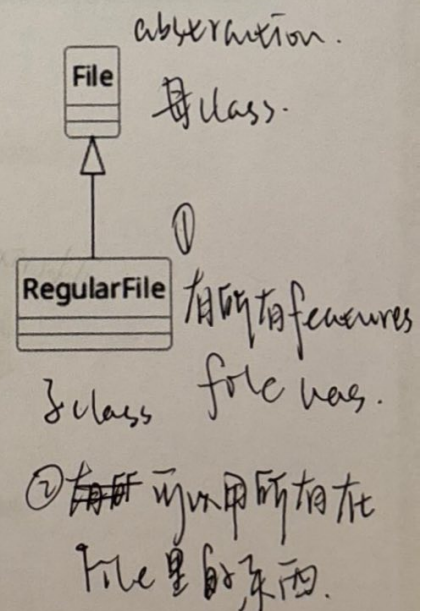
© Jarmo Hurri

10/19

Inheritance

properties. 属性继承.

- relationship indicating that the class which inherits
 - "is a" subtype of another class, and/or
 - is able to reuse the code of another class
- static: does not change over lifetime of program
- RegularFile is a File
- File is a superclass (or parent) of RegularFile
- RegularFile is a subclass of File
- RegularFile *extends* File
- other examples
 - Car is a Vehicle
 - Manager is an Employee



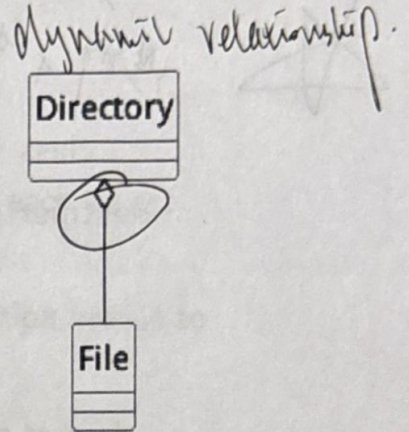
③ static. 静态的.

© Jarmo Hurri

11/19

Aggregation

- relationship indicating that one class
 - "has" (or "contains") objects of another class, or
 - consists of objects of another class
- Directory contains Files
- dynamic: Files may change
- does not automatically imply ownership
 - Files can also be contained in another Directory (links)
- other examples
 - Company has Employees
 - Laptop has a Keyboard
 - InEquality contains Expressions

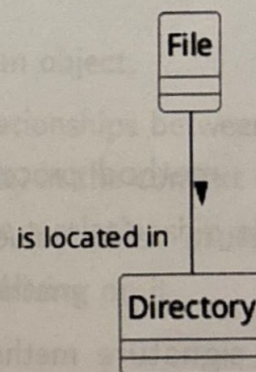


© Jarmo Hurri

12/19

Dependency

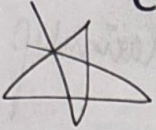
- general relationship indicating that one class "uses" another class for something else than inheritance or aggregation
- File is located in (multiple objects of type) Directory
- relationship not unique: depends on chosen representation
- dynamic
- other examples
 - Operating system uses Disk (for permanent storage)
 - Factory uses Security company
 - Author uses Typesetting system (to create book layout)



© Jarmo Hurri

13/19

Central OOP terminology



class + attribute

class template / blueprint for a new data type

identifier name given by programmer to a specific program

entity

- entities: e.g. method, class, variable
- examples: File (name of class), price (name of variable)

primitive low-level building blocks available in a programming language (e.g. statement)

attribute (in Python) any value associated with object referred to using dot notation (e.g. obj.attr)

Terminology related to functions



method procedure associated with a class or a specific object

return value value (possibly) passed back by the method on method exit

signature method identification containing name and parameters (with or without their types)

constructor method creating an object

- name identical to name of class

Different types of variables



class
def
→

(in an object)

instance variable named storage location unique to each object
(instance of an class)

class
→

static variable / class variable named storage location unique to
each class

parameter (variable) named storage location unique to each
method call

local variable named storage location unique to each execution of
a code block (e.g., in a method)

- bound inside block

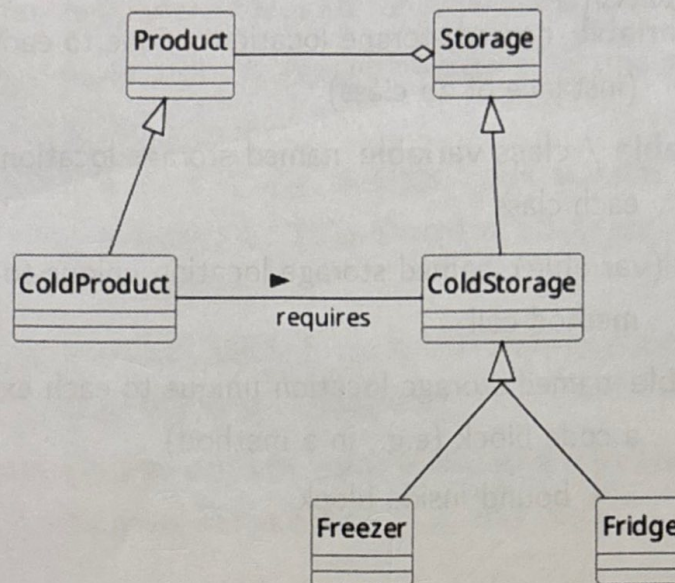
Homework 1

1. Outline the difference between a class and an object.
2. For each row, outline two possible UML relationships between the classes in the given context. For example, in the context of transportation, classes Car, Road could have a relationship of aggregation Road has a number of Car s driving on it
dependency Car uses a number of Road s

context	classes
transportation	Train, Passenger
school	Student, Course
interior design	Table, Furniture
family	Parent, Child
library	Book, Literature

Homework 2

Interpret the UML diagram below



Homework 3

Consider the code on the right.

1. State the variable type of each identifier *x*.
2. Which variable type is *not* present in the code?
3. What entity does each identifier *Point* refer to?
4. At which location of code is the function called automatically?
 - 4.1 `__init__()`
 - 4.2 `__str__()`

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __str__(self):
        s = f'({self.x}, {self.y})'
        return s

def origin():
    x = Point(0, 0)
    return x

x = origin()
print(x)
```